
ASSIGNMENT 21

Name: Om Prashant Kulawade

College: Zeal Polytechnic, Narhe, Pune

Branch: Artificial Intelligence and Machine Learning (AIML)

Domain: AIML (Artificial Intelligence and Machine Learning)

<https://github.com/omkulawade03/Frontend-Session-Assignments.git>

Q1. (Setup and Libraries)

Create a new Python file named `app.py`.

Import the required libraries: `streamlit`, `pandas`, and `joblib`.

Write comments explaining the purpose of each library.

Program Code:

#Questions 01

```
app.py
1  # =====
2  # Q1. Import Required Libraries
3  # =====
4
5  # Streamlit is used to create the web application
6  import streamlit as st
7
8  # Pandas is used for data manipulation and DataFrame creation
9  import pandas as pd
10
11 # Joblib is used to load the saved machine learning model
12 import joblib
13
```

OUTPUT SCREENSHOT:

```
PS C:\Users\Asus\OneDrive\Desktop\Fronted_learning\Ford_Price_Predictor> & 'C:\Users\Asus\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'C:\Users\Asus\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '63757' '--' 'C:\Users\Asus\OneDrive\Desk
top\Fronted_learning\Ford_Price_Predictor\app.py'
PS C:\Users\Asus\OneDrive\Desktop\Fronted_learning\Ford_Price_Predictor>
```

Q2. (Loading Model and Preprocessing Objects)

Write code to load the saved files:

LR_model.pkl (trained model)

scaler.pkl (StandardScaler)

columns.pkl (encoded column names)

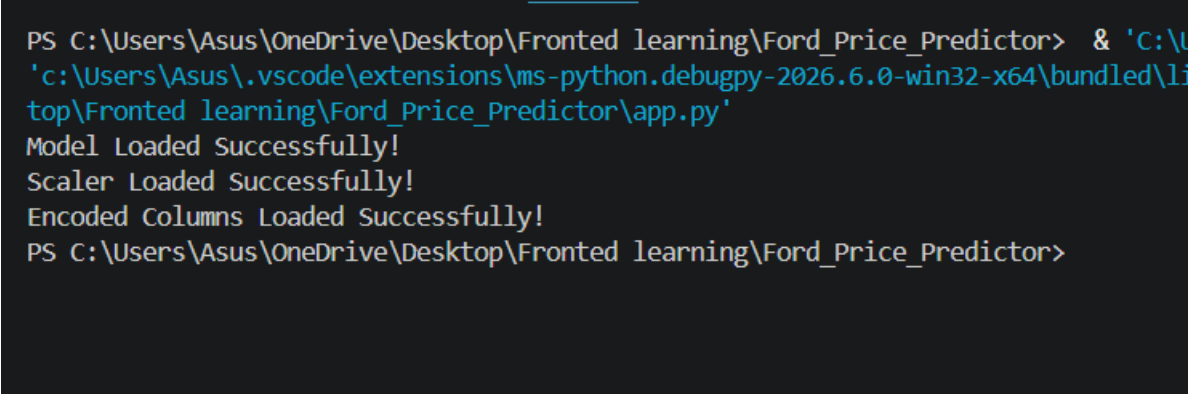
Use `joblib.load()` and store them in variables `model`, `scaler`, and `encoded_columns`.

Program Code:

#Questions 02

```
14
15  # =====
16  # Q2. Load Trained Model and Preprocessing Objects
17  # =====
18
19  # Load the trained Linear Regression model
20  model = joblib.load("LR_model.pkl")
21  print("Model Loaded Successfully!")
22
23  # Load the StandardScaler object
24  scaler = joblib.load("scaler.pkl")
25  print("Scaler Loaded Successfully!")
26
27  # Load the encoded column names
28  encoded_columns = joblib.load("columns.pkl")
29  print("Encoded Columns Loaded Successfully!")
30
```

OUTPUT SCREENSHOT:



```
PS C:\Users\Asus\OneDrive\Desktop\Fronted learning\Ford_Price_Predictor> & 'C:\U
'c:\Users\Asus\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\l
top\Fronted learning\Ford_Price_Predictor\app.py'
Model Loaded Successfully!
Scaler Loaded Successfully!
Encoded Columns Loaded Successfully!
PS C:\Users\Asus\OneDrive\Desktop\Fronted learning\Ford_Price_Predictor>
```

Q3. (Page Configuration)

Configure the Streamlit page using `st.set_page_config()`.

Set `page_title` as "Ford Car Price Predictor".

Set `layout` as "centered".

Add a comment explaining why these settings are used.

Program Code:

#Questions 03

```
33 # =====
34 # Q3. Configure Streamlit Page
35 # =====
36
37 # Set the page title and center the layout
38 st.set_page_config(
39     page_title="Ford Car Price Predictor",
40     layout="centered"
41 )
42
```

OUTPUT SCREENSHOT:

```
PS C:\Users\Asus\OneDrive\Desktop\Fronted Learning\Ford_Price_Predictor> & 'C:\Users\Asus\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'c:\Users\Asus\.vscode\extensions\ms-python.debugpy.2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62670' '-.' 'C:\Users\Asus\OneDrive\Desk
top\Fronted Learning\Ford_Price_Predictor\app.py'
Model Loaded Successfully!
Scaler Loaded Successfully!
Encoded Columns Loaded Successfully!
2026-07-18 13:02:37.789 WARNING: streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'Mainthread': missing ScriptRunContext! This wa
rning can be ignored when running in bare mode.
PS C:\Users\Asus\OneDrive\Desktop\Fronted Learning\Ford_Price_Predictor>
```

Q4. (Title and Description)

Display:

A main title using st.title().

A short description using st.write().

Example: "Enter the car details below to predict its selling price."

Program Code:

#Questions 04

```
43
44 # =====
45 # Q4. App Title and Description
46 # =====
47
48 st.title("Ford Car Price Predictor")
49
50 st.write("Enter the car details below to predict its selling price.")
51
```

OUTPUT SCREENSHOT:

A screenshot of a web application titled "Ford Car Price Predictor". The title is in a large, bold, white font. Below it, in a smaller white font, is the instruction "Enter the car details below to predict its selling price." The background is a solid dark blue.

Ford Car Price Predictor

Enter the car details below to predict its selling price.

Q5. (Numerical Input Fields)

Create input fields using `st.number_input()` for the following:

Manufacturing Year (year)

Mileage

Road Tax (tax)

MPG

Engine Size

Set appropriate `min_value`, `max_value`, and `default value`.

Program Code:

#Questions 05

```

54 # =====
55 # Q5. Numerical Input Fields
56 # =====
57
58 # Manufacturing Year
59 year = st.number_input(
60     "Manufacturing Year",
61     min_value=1996,
62     max_value=2025,
63     value=2018
64 )
65
66 # Mileage
67 mileage = st.number_input(
68     "Mileage",
69     min_value=0,
70     max_value=300000,
71     value=30000
72 )
73
74 # Road Tax
75 tax = st.number_input(
76     "Road Tax",
77     min_value=0,
78     max_value=600,
79     value=150
80 )
81
82 # MPG
83 mpg = st.number_input(
84     "MPG",
85     min_value=10.0,
86     max_value=100.0,
87     value=50.0
88 )
89
90 # Engine Size
91 engine_size = st.number_input(
92     "Engine Size",
93     min_value=0.8,
94     max_value=5.0,
95     value=1.5
96 )
97

```

OUTPUT SCREENSHOT:

The screenshot shows a web application titled "Ford Car Price Predictor" on a dark background. Below the title is a subtitle: "Enter the car details below to predict its selling price." There are five input fields, each with a label and a numeric value, and each field has minus and plus buttons for adjustment. The inputs are: Manufacturing Year (2018), Mileage (30000), Road Tax (150), MPG (50.00), and Engine Size (1.50).

Ford Car Price Predictor

Enter the car details below to predict its selling price.

Manufacturing Year

2018 - +

Mileage

30000 - +

Road Tax

150 - +

MPG

50.00 - +

Engine Size

1.50 - +

Q6. (Categorical Input using Dropdowns)

Create dropdowns using `st.selectbox()` for:

Transmission (Automatic, Manual, Semi-Auto)

Fuel Type (Petrol, Diesel, Hybrid, etc.)

Add comments explaining the advantage of using `selectbox`.

Program Code:

#Questions 06

```
100
101 # =====
102 # Q6. Categorical Inputs using Dropdown
103 # =====
104
105 # Dropdown for Transmission
106 transmission = st.selectbox(
107     "Transmission",
108     ["Automatic", "Manual", "Semi-Auto"]
109 )
110
111 # Dropdown for Fuel Type
112 fuel_type = st.selectbox(
113     "Fuel Type",
114     ["Petrol", "Diesel", "Hybrid", "Electric", "Other"]
115 )
116
```

OUTPUT SCREENSHOT:

The screenshot displays two dropdown menus on a dark background. The first menu, labeled 'Transmission', has 'Automatic' selected and shows a list of options: 'Automatic', 'Manual', and 'Semi-Auto'. The second menu, labeled 'Fuel Type', has 'Petrol' selected and shows a list of options: 'Petrol', 'Diesel', 'Hybrid', 'Electric', and 'Other'.

Transmission

Automatic

Automatic

Manual

Semi-Auto

Fuel Type

Petrol

Petrol

Diesel

Hybrid

Electric

Other

Q7. (Text Input and Predict Button)

Take car model name using `st.text_input()`.

Create a Predict Price button using `st.button()`.

Program Code:

#Questions 07

```

119 # =====
120 # Q7. Text Input and Predict Button
121 # Take car model name and create Predict Price button.
122 # =====
123
124 # Car Model Selection
125 car_model = st.selectbox(
126     "Car Model",
127     [
128         "B-MAX",
129         "C-MAX",
130         "EcoSport",
131         "Edge",
132         "Escort",
133         "Fiesta",
134         "Focus",
135         "Fusion",
136         "Galaxy",
137         "Grand C-MAX",
138         "KA",
139         "Ka+",
140         "Kuga",
141         "Mondeo",
142         "Mustang",
143         "Puma",
144         "Ranger",
145         "S-MAX",
146         "Streetka",
147         "Torneo Connect",
148         "Torneo Custom",
149         "Transit Tourneo"
150     ]
151 )
152
153 # Predict Button
154 predict = st.button("Predict Price")

```

OUTPUT SCREENSHOT:

The image displays two screenshots of a web application interface for selecting a car model. The top screenshot shows a dropdown menu labeled 'Car Model' with 'B-MAX' selected. Below the dropdown, a list of car models is visible: Edge, Escort, Fiesta, Focus, Fusion, Galaxy, and Grand C-MAX. The bottom screenshot shows the same interface, but with 'Fiesta' selected in the dropdown menu. Below the dropdown, there is a button labeled 'Predict Price'.

Q8. (Creating Input DataFrame & Encoding)

When the Predict button is clicked:

Create a pandas DataFrame from user inputs.

Perform One-Hot Encoding using `pd.get_dummies()`.

Align the columns with the training columns (`encoded_columns`).

Program Code:

#Questions 08

```
132
133 # =====
134 # Q8. Create DataFrame and Perform One-Hot Encoding
135 # =====
136
137 if predict:
138
139     # Create dictionary from user input
140     input_data = {
141         "model": car_model,
142         "year": year,
143         "transmission": transmission,
144         "mileage": mileage,
145         "fuelType": fuel_type,
146         "tax": tax,
147         "mpg": mpg,
148         "engineSize": engine_size
149     }
150
151     # Convert dictionary into DataFrame
152     input_df = pd.DataFrame([input_data])
153
154     # Display Input Data
155     st.subheader("Input Data")
156     st.dataframe(input_df)
157
158     # Perform One-Hot Encoding
159     input_encoded = pd.get_dummies(input_df)
160
161     # Match columns with training dataset
162     input_encoded = input_encoded.reindex(
163         columns=encoded_columns,
164         fill_value=0
165     )
166
167     # Display Encoded Data
168     st.subheader("Encoded Input Data")
169     st.dataframe(input_encoded)
170
```

OUTPUT SCREENSHOT:

Car Model

Focus

Predict Price

Input Data

	model	year	transmission	mileage	fuelType	tax	mpg	engineSize
0	Focus	2018	Automatic	30000	Petrol	150	50	1.5

Encoded Input Data

	year	mileage	tax	mpg	engineSize	model_ B-MAX	model_ C-MAX	model_ EcoSport	model_
0	2018	30000	150	50	1.5	0	0	0	

Q9. (Feature Scaling and Prediction)

Identify numerical columns and apply the loaded scaler.

Make prediction using the loaded model.

Display the predicted price using `st.success()` with proper formatting (£ and 2 decimal places).

Program Code:

#Questions 09

```
196
197 # =====
198 # Q9. Feature Scaling and Prediction
199 # =====
200
201     # Numerical columns used during training
202     numerical_columns = [
203         "year",
204         "mileage",
205         "tax",
206         "mpg",
207         "engineSize"
208     ]
209
210     # Apply StandardScaler
211     input_encoded[numerical_columns] = scaler.transform(
212         input_encoded[numerical_columns]
213     )
214
215     # Predict Selling Price
216     prediction = model.predict(input_encoded)
217
218     # Display Prediction
219     st.success(f"Predicted Price: £{prediction[0]:,.2f}")
220
```

OUTPUT SCREENSHOT:

Car Model

Focus

Predict Price

Input Data

	model	year	transmission	mileage	fuelType	tax	mpg	engineSize
0	Focus	2018	Automatic	30000	Petrol	150	50	1.5

Encoded Input Data

	year	mileage	tax	mpg	engineSize	model_B-MAX	model_C-MAX	model_EcoSport	model_Focus
0	2018	30000	150	50	1.5	0	0	0	1

Predicted Price: £12,043.44

Q10. (Mini Project - Complete Streamlit App)

Build the complete Ford Car Price Prediction web app by combining all the above steps.

Requirements:

Clean and well-commented code.

Proper error handling (if possible).

Nice layout using Streamlit elements.

Run the app locally using streamlit run app.py and take screenshots of the working app.

Program Code:

#Questions 10

```
223
224 # =====
225 # Q10. Mini Project - Complete Streamlit App
226 # =====
227
228 st.write("---")
229
230 st.subheader("Assignment 21")
231
232 st.write("Ford Car Price Predictor Web Application")
233
234 st.write("✓ Model Loaded")
235
236 st.write("✓ User Input Taken")
237
238 st.write("✓ Price Predicted Successfully")
239
240 st.success("Application Running Successfully!")
```

OUTPUT SCREENSHOT:

